Speaker Notes

LA Kelp Detection Prototype
Weekly Progress Report

Ekaba Bisong

February 25, 2026

Contents

1 Slide 1: Title Slide **3**

2 Slide 2: Outline **3**

3 Slide 3: Week 1 – Committee Meeting & Project Setup **3**
3.1 Key Points to Emphasize . . . 3
3.2 What is L_{R-I}? . . . 3

4 Slide 4: Week 1 – Prototype Architecture **3**
4.1 The Four Heuristics . . . 3
4.2 Infrastructure Decisions . . . 4

5 Slide 5: Week 1 – Next Steps **4**

6 Slide 6: Week 2 – Prototype Implementation Complete **4**
6.1 What Are the Baselines? . . . 4

7 Slide 7: Week 2 – Baseline Results **4**
7.1 Understanding the Numbers . . . 4
7.2 Key Observations . . . 5
7.3 The Oracle Gap . . . 5

8 Slide 8: Week 2 – Experiment 1 (Binary Reward, Failed) **5**
8.1 Binary Reward Definition . . . 5
8.2 Why It Failed . . . 5

9 Slide 9: Week 2 – Fix with Continuous Reward **6**
9.1 Continuous Reward Definition . . . 6
9.2 The Solution . . . 6
9.3 Why This Works . . . 6

10 Slide 10: Week 2 – Experiment 2 (Continuous, $\alpha = 0.1$) **6**
10.1 The Result . . . 6
10.2 What is Entropy? . . . 7
10.3 Why Fast Convergence is Bad . . . 7

11 Slide 11: Week 2 – Experiment 3 (Continuous, $\alpha = 0.01$) **7**
11.1 Proper Learning Dynamics . . . 7
11.2 Reading the Probability Table . . . 8

12 Slide 12: Week 2 – Experiment Comparison **8**
12.1 Table Breakdown: What Each Row Means . . . 8
12.2 Key Insight: Same Destination, Different Journey . . . 8
12.3 Trade-offs . . . 8
12.4 Gap to Oracle . . . 8

13 Slide 13: Week 2 – Key Learnings **9**
13.1 Technical Lessons . . . 9
13.2 Next Steps . . . 9
13.3 Broader Implications . . . 9

14 Slide 14: Next Steps – Prototype Refinement **9**
14.1 Prototype Refinement . . . 9

15 Slide 15: Next Steps – LA Variant Comparison **9**
15.1 Available Algorithms . . . 9
15.2 Talking Points . . . 10
15.3 Experiment Plan . . . 10

16 Summary: What to Remember **10**

Slide 1: Title Slide

- This presentation covers the progress on building a Learning Automata prototype for kelp detection using Sentinel-2 satellite imagery.
- The prototype implements the core ideas from the thesis proposal: using LA to adaptively select detection heuristics.
- We'll cover two weeks of work: project setup and initial experiments.

Slide 2: Outline

- Week 1 focused on setup after the committee meeting.
- Week 2 focused on implementation and running first experiments.
- The key finding: the prototype works, but hyperparameter tuning matters.

Slide 3: Week 1 – Committee Meeting & Project Setup

Key Points to Emphasize

- The committee meeting on February 5th provided critical guidance: **build a full but simple prototype first**.
- "Full" means end-to-end: data loading, heuristics, LA, evaluation.
- "Simple" means spectral heuristics only—no deep learning yet.
- Important scope clarification: we're doing kelp **mapping** (binary segmentation), not full ecological monitoring (species identification, health assessment, etc.).
- The Global Policy approach uses a single L_{R-I} automaton that learns one probability distribution over heuristics for the entire dataset.

What is L_{R-I}?

- L_{R-I} stands for "Linear Reward-Inaction."
- **Reward**: When the selected heuristic performs well, increase its probability and decrease others.
- **Inaction**: When performance is poor, do nothing (don't update probabilities).
- This is a conservative scheme—it only reinforces good outcomes.

Slide 4: Week 1 – Prototype Architecture

The Four Heuristics

- **NDVI** (Normalized Difference Vegetation Index): Classic vegetation index using NIR and Red bands. Works well for terrestrial vegetation, less tuned for marine.
- **FAI** (Floating Algae Index): Designed specifically for detecting floating vegetation in water. Uses NIR, Red, and SWIR bands.

- **GNDVI** (Green NDVI): Uses Green band instead of Red. Often better for aquatic vegetation.
- **Ensemble**: Takes majority vote across the other three heuristics.

Infrastructure Decisions

- `config.yaml`: Single source of truth for all experiment parameters (learning rate, reward type, number of epochs, etc.).
- `Makefile`: Reproducible experiment execution with `make run` and `make baselines`.
- Experiment tracker auto-logs results, plots, and configuration for each run.

Slide 5: Week 1 – Next Steps

- The immediate tasks were to build the infrastructure and implement all components.
- Success criteria are concrete and measurable:
  1. The LA must update probabilities correctly (mechanical correctness).
  2. Probabilities should converge toward better heuristics (learning signal).
  3. LA should beat random selection (practical value).
- If we can't beat random selection, the LA isn't providing value.

Slide 6: Week 2 – Prototype Implementation Complete

- All core components are now implemented and working.
- The data loader handles 10-band Sentinel-2 tiles (coastal aerosol, blue, green, red, vegetation red edge bands, NIR, SWIR).
- The reward function supports both binary (threshold-based) and continuous (raw IoU) modes.
- We have three baselines for comparison: Random, Fixed, and Oracle.

What Are the Baselines?

- **Random**: Select a heuristic uniformly at random for each tile. This is the **lower bound**—if LA can't beat this, it's useless.
- **Fixed**: Always use the same heuristic for every tile. One fixed baseline per heuristic.
- **Oracle**: Try all heuristics, pick the best one per tile (using ground truth). This is the **upper bound**—perfect hindsight selection.

Slide 7: Week 2 – Baseline Results

Understanding the Numbers

- IoU (Intersection over Union) measures overlap between prediction and ground truth. IoU = 1 is perfect, IoU = 0 is no overlap.

- Our IoU values are low (0.007–0.076) because kelp covers only a small fraction of each tile. This is **class imbalance**.
- Even with low absolute IoU, the **relative ranking** is meaningful.

Key Observations

- **Oracle (0.0765)**: Best possible. Shows there's room for adaptive selection.
- **GNDVI (0.0699)**: Best single heuristic. Green-based index works well for kelp.
- **Ensemble (0.0695)**: Close second. Majority voting is robust.
- **NDVI (0.0533)**: Middle performer. Standard vegetation index.
- **Random (0.0445)**: Our floor. LA must beat this.
- **FAI (0.0074)**: Worst performer. Despite being designed for floating algae, it fails here—possibly due to threshold tuning or water conditions.

The Oracle Gap

- Gap between Oracle (0.0765) and best Fixed (0.0699) is about 9%.
- This gap represents the potential gain from adaptive selection.
- If LA can close this gap, it's learning something useful.

Slide 8: Week 2 – Experiment 1 (Binary Reward, Failed)

Binary Reward Definition

From `reward.py:126-129`:

```
if self.reward_type == "binary":
    reward = 1.0 if iou > self.iou_threshold else 0.0
```

- Binary reward is a threshold-based signal:
  - If IoU $> 0.3 \rightarrow$ reward $=$ **1** (success)
  - If IoU $\leq 0.3 \rightarrow$ reward $=$ **0** (failure)
- The threshold 0.3 was chosen arbitrarily—a "reasonable" value.

Why It Failed

Your actual IoU values ranged from **0.007 to 0.076** (kelp is sparse). Since **no tile ever exceeded 0.3**, the automaton received:

Tile	IoU	Reward
Tile 1	0.05	0
Tile 2	0.07	0
Tile 3	0.01	0
...	...	0

- Every heuristic got reward = 0, so the L_{R-I} automaton had **no learning signal**.
- Probability updates became random drift.
- It converged to FAI **by chance**—the worst heuristic!
- This is a classic failure mode: **threshold miscalibration**.

Slide 9: Week 2 – Fix with Continuous Reward

Continuous Reward Definition

From `reward.py:128-129`:

```
elif self.reward_type == "continuous":
    reward = iou # Use IoU directly
```

The Solution

- Continuous reward uses IoU directly: $r = \text{IoU}$.
- Now every tile provides a gradient signal, even with low IoU values.

  Now the automaton sees meaningful differences:

Heuristic	Reward (avg)
FAI	≈ 0.007
GNDVI	≈ 0.07

- Even though both are "low," the **10× difference** provides a gradient signal.
- The automaton can now learn that GNDVI/Ensemble are better than FAI.
- The automaton can distinguish between "bad" and "very bad."

Why This Works

- Binary reward is like pass/fail grading—you lose information.
- Continuous reward is like percentage grading—you preserve ordering.
- For problems with extreme class imbalance, continuous rewards are often necessary.

Slide 10: Week 2 – Experiment 2 (Continuous, $\alpha = 0.1$)

The Result

- With continuous reward, LA correctly converged to Ensemble—one of the best heuristics!
- **But**: Convergence was too fast. By epoch 2, Ensemble had 99.9999% probability.
- Entropy dropped to 0, meaning no exploration after epoch 2.

What is Entropy?

Entropy measures how "spread out" the probability distribution over heuristics is.

Formula: For a probability distribution $p = [p_1, p_2, p_3, p_4]$ over 4 heuristics:

$$H(p) = -\sum_{i=1}^{4} p_i \log(p_i)$$

What It Means:

Entropy	Distribution	Interpretation
High (≈1.39)	[25%, 25%, 25%, 25%]	Uniform — maximum exploration
Medium (≈1.52)	[20%, 0%, 41%, 39%]	Some preference, still exploring
Low (≈0.1)	[1%, 0%, 0%, 99%]	Strong preference, little exploration
Zero (0.0)	[0%, 0%, 0%, 100%]	Converged — no exploration at all

In Your Experiments:

- **Exp 2 (α=0.1)**: Entropy dropped to **0.0** by epoch 2 — the automaton "locked in" on Ensemble with 100% probability. No more exploration.
- **Exp 3 (α=0.01)**: Entropy stayed at **1.52** after 10 epochs — GNDVI (41%) and Ensemble (39%) are still competing. The automaton is still exploring.

Why It Matters:

- Low entropy too early = **premature convergence**. The automaton commits before gathering enough evidence.
- Higher entropy = continued learning and potential to find better solutions.

Why Fast Convergence is Bad

- The automaton "locked in" before exploring alternatives.
- It converged to Ensemble, but GNDVI is actually slightly better (0.0699 vs 0.0695).
- With more exploration, it might have found GNDVI.
- Also, we can't trust results from only 1 epoch of real learning.

Slide 11: Week 2 – Experiment 3 (Continuous, $\alpha = 0.01$)

Proper Learning Dynamics

- Learning rate $\alpha = 0.01$ (10x smaller than before).
- Now we see gradual convergence over 10 epochs.
- FAI is correctly eliminated (probability drops to 0%).
- GNDVI and Ensemble compete for dominance (41% vs 39% by epoch 10).
- Entropy = 1.52 means ongoing exploration.

Reading the Probability Table

- Epoch 1: Roughly uniform (37%, 3%, 27%, 34%). FAI already penalized.
- Epoch 4: NDVI temporarily leads (44%), FAI nearly gone (0.03%).
- Epoch 7: Ensemble surges (53%), competition continues.
- Epoch 10: GNDVI edges ahead (41%), Ensemble close behind (39%).
- This matches our baseline ranking: GNDVI > Ensemble > NDVI $\gg$ FAI.

Slide 12: Week 2 – Experiment Comparison

Table Breakdown: What Each Row Means

Metric	Exp 2	Exp 3	Explanation
Convergence	Epoch 2	10 epochs	How fast it "decided"
Final Entropy	0.0	1.52	0 = no exploration; 1.52 = still exploring
Best Heuristic Prob	100%	41%	Exp 2 went "all in"; Exp 3 hedged
Test IoU	0.0719	0.0711	Nearly identical performance
vs Random	+0.0151	+0.0144	Both ∼34% better than random
Gap to Oracle	6.0%	7.0%	How far from perfect (0.0765)

Key Insight: Same Destination, Different Journey

- Both experiments achieved similar final performance (∼0.071 IoU).
- But Exp 2 (high α) committed immediately, while Exp 3 (low α) explored longer.
- Exp 2 got lucky—it happened to lock onto a good heuristic early.
- Exp 3 is more trustworthy—the gradual convergence shows the automaton actually **learned** the ranking.

Trade-offs

	High α (0.1)	**Low α (0.01)**
Pro	Fast convergence	Robust, explores alternatives
Con	Risk of premature lock-in	Slower to converge

- For **research**, prefer low α to observe learning dynamics.
- For **production** with time constraints, higher α may be acceptable.

Gap to Oracle

- LA achieves 94% of Oracle performance.
- The remaining 6-7% gap suggests room for improvement.
- Possible improvements: per-tile context, more heuristics, better reward shaping.

Slide 13: Week 2 – Key Learnings

Technical Lessons

- **Lesson 1**: Binary reward requires threshold calibration. Know your data distribution before setting thresholds.
- **Lesson 2**: Continuous reward is more robust for imbalanced problems.
- **Lesson 3**: Learning rate controls exploration/exploitation trade-off.
- **Lesson 4**: L_{R-I} successfully learns heuristic quality rankings—the core hypothesis is validated!

Next Steps

- Run more epochs (20-50) to see full convergence behavior.
- Try even smaller α (0.001) for more gradual learning curves.
- Add ε-greedy exploration: with probability ε, select randomly instead of following the learned distribution. This prevents probability collapse.

Broader Implications

- The prototype validates the core thesis idea: LA can learn to select appropriate heuristics.
- Next phase: move from Global Policy (one distribution for all tiles) to Contextual Policy (different distributions based on tile features).
- Eventually: incorporate ML-based heuristics alongside spectral ones.

Slide 14: Next Steps – Prototype Refinement

Prototype Refinement

- Run more epochs (20–50) to see if distribution fully converges or keeps oscillating.
- Try $\alpha = 0.001$ for even more gradual learning curves.
- Add ε-greedy exploration: with probability ε, select randomly. Prevents probability collapse to 0/100%.

Slide 15: Next Steps – LA Variant Comparison

Available Algorithms

All implemented in `automaton.py`:

Algorithm	Description	Expected Behavior
L_{R-I}	Reward only, ignore penalty	Conservative, stable
L_{R-P}	Reward and penalty updates	Faster but noisier
Pursuit	Estimates reward, pursues best	10–20x faster convergence
Discretized Pursuit	Finite probability levels	Even faster
Estimator (SERI)	Bayesian estimation + pursuit	Fastest ε-optimal
VSLA	Adaptive α based on entropy	Auto-balances explore/exploit

Talking Points

- $\mathbf{L}_{R-I}$ (what we tested): Only updates on positive reward. Conservative—won't over-react to noise.
- $\mathbf{L}_{R-P}$: Also penalizes bad actions. Faster learning but can oscillate in noisy environments.
- **Pursuit**: Maintains reward *estimates* for each action, then "pursues" the best estimated action. Much faster convergence (10–20x) because it uses information more efficiently.
- **Discretized Pursuit**: Uses finite probability levels instead of continuous. Can converge even faster.
- **Estimator (SERI)**: Bayesian version of Pursuit. Maintains Beta distribution posteriors. Recognized as one of the fastest ε-optimal algorithms.
- **VSLA**: Automatically adjusts learning rate based on entropy. High entropy $\rightarrow$ high α (explore). Low entropy $\rightarrow$ low α (exploit).

Experiment Plan

- Run all 6 algorithms on the same kelp detection task.
- Compare: convergence speed, final IoU, stability, entropy dynamics.
- Hypothesis: Pursuit/Estimator will converge faster but L_{R-I} may be more stable.

Summary: What to Remember

1. **The prototype works.** LA successfully learns to favor better heuristics.
2. **Reward design matters.** Binary reward failed due to threshold miscalibration; continuous reward succeeded.
3. **Hyperparameters matter.** Learning rate α controls convergence speed and exploration.
4. **Baselines provide context.** Random (floor), Oracle (ceiling), Fixed (single-heuristic performance).
5. **LA outperforms random selection.** This validates the adaptive selection approach.